CSE 713 — Artificial Intelligence

STRIPS + Partial-Order Planning (Block World)

Complete Past Paper Solutions: 2020–2024

Source: *Planning.pdf* (50 slides) + *plan2.pdf* (38 slides)

Every question from 2020–2024 included. Zero omissions.

Contents

1 2024 — Q5 (9 marks) 4
1.1 Part (a) — STRIPS + Household Robot Action Schema [4 marks] 4
1.2 Part (b) — POP for Standard Block World [5 marks] 5

2 2023 — Q5 (9 marks) 7
2.1 Part (a) — STRIPS + Air Cargo Action Schema [4 marks] 7
2.2 Part (b) — POP for Standard Block World [5 marks] 7

3 2022 — A-3 (9 marks) 9
3.1 Part (a) — Block World Operator Preconditions [2 marks] 9
3.2 Part (b) — Block World with STRIPS / POP [5 marks] 9
3.3 Part (c) — Short Notes: Generative AI [2 marks] 10

4 2021 — Q6(b): Sussman Anomaly [4 marks] 12

5 2020 — B-Q6 (estimated 9 marks) 14

Topic Overview (from Slides)

STRIPS + POP — What to Know Cold

STRIPS (Stanford Research Institute Problem Solver):

- Action = Preconditions + Add-list + Delete-list
- State = conjunction of positive ground literals
- Closed World Assumption: anything not stated is false
- Solves the Frame Problem: only explicit effects change the state

Partial-Order Planning (POP):

- Plan = (A, O, L): set of actions, ordering constraints, causal links
- Causal link: $A_p \xrightarrow{Q} A_c$ means A_p achieves condition Q for A_c
- Threat: action A_t threatens link (A_p, Q, A_c) if A_t deletes Q and could fall between A_p and A_c
- Resolution: **Demotion** ($A_t \prec A_p$) or **Promotion** ($A_c \prec A_t$)
- Solution: no open conditions, no threats, consistent ordering
- POP is **sound and complete**

Special actions for POP:

- A_0: no preconditions; effects = initial state; must be **first**
- A_∞: no effects; preconditions = goal; must be **last**

Block World STRIPS Operators (Memorise These)

4 Standard Block World Operators

Action	Preconditions	Add	Delete
PICKUP(b)	ONTABLE(b), CLEAR(b), ARMEMPTY	HOLDING(b)	ONTABLE(b), CLEAR(b), ARMEMPTY
PUTDOWN(b)	HOLDING(b)	ONTABLE(b), CLEAR(b), ARMEMPTY	HOLDING(b)
UNSTACK(b, c)	ON(b,c), CLEAR(b), ARMEMPTY	HOLDING(b), CLEAR(c)	ON(b,c), CLEAR(b), ARMEMPTY
STACK(b, c)	HOLDING(b), CLEAR(c)	ON(b,c), ARMEMPTY, CLEAR(b)	HOLDING(b), CLEAR(c)

Memory rule: UNSTACK = pick up a block from another block. PICKUP = pick up from table. STACK = place on a block. PUTDOWN = place on table.

The Standard 4-Block World (2022, 2023, 2024 — appears verbatim every year)

Initial State: ON(B,A), ONTABLE(A), ONTABLE(C), ONTABLE(D), ARMEMPTY, CLEAR(B), CLEAR(C), CLEAR(D)
(A is NOT clear — B is sitting on top of it.)

Goal State: ON(C,A), ON(B,D), ONTABLE(A), ONTABLE(D)

Total-order solution: UNSTACK(B,A) → STACK(B,D) → PICKUP(C) → STACK(C,A)

2024 — Q5 (9 marks)

Part (a) — STRIPS + Household Robot Action Schema [4 marks]

2024 Q5(a) — STRIPS Definition + Action Schema

What is STRIPS? How are actions represented using STRIPS action schema? Develop a planning problem using STRIPS defining one action for a **Household Robot** (cleaning a room, turning on appliances, fetching objects) with Preconditions, Add-effects, and Delete-effects.

Solution

STRIPS (Stanford Research Institute Problem Solver) is a planning framework representing the world as a set of positive ground literals (propositions). An action in STRIPS is an **action schema** consisting of three components:

1. **Preconditions**: a conjunction of positive, function-free literals that must hold in the current state before the action can execute.
2. **Add-list (Add-effects)**: propositions that become **true** after the action.
3. **Delete-list (Delete-effects)**: propositions that become **false** after the action.

Key property (Frame Problem solution): Only propositions explicitly in the add/delete lists change; everything else persists.

Household Robot — Three Action Schemas:

`Action: CLEAN_ROOM(room)`
Preconditions: {IN(robot, room), DIRTY(room)}
Add-effects: {CLEAN(room)}
Delete-effects: {DIRTY(room)}

`Action: TURN_ON(appliance, loc)`
Preconditions: {IN(robot, loc), AT(appliance, loc), OFF(appliance)}
Add-effects: {ON(appliance)}
Delete-effects: {OFF(appliance)}

`Action: FETCH(obj, loc)`
Preconditions: {IN(robot, loc), LOCATED(obj, loc)}
Add-effects: {HOLDING(obj)}
Delete-effects: {LOCATED(obj, loc)}

(Any one of the above three is sufficient for full marks — the question asks for "one action.")

Part (b) — POP for Standard Block World [5 marks]

2024 Q5(b) — Partial-Order Planning

Block World (Start / Goal as above). Using Partial-Order Planning (POP):

1. Construct the minimal POP plan.
2. List all causal links and ordering constraints.
3. Explain how POP avoids backtracking compared to total-order planning.

Solution

Step 1: Actions required and their roles.

- Goal **ON(C,A)** requires STACK(C,A). Its preconditions: HOLDING(C) and CLEAR(A).
- **CLEAR(A)** is not in the initial state (B is on A). Need UNSTACK(B,A) first.
- UNSTACK(B,A) produces HOLDING(B). Use it immediately: STACK(B,D) achieves goal ON(B,D).
- STACK(B,D) restores ARMEMPTY $\Rightarrow$ PICKUP(C) can run next.
- PICKUP(C) produces HOLDING(C) needed by STACK(C,A).

Plan Steps:

$$S_1 = \text{UNSTACK}(B, A), \quad S_2 = \text{STACK}(B, D), \quad S_3 = \text{PICKUP}(C), \quad S_4 = \text{STACK}(C, A)$$

Step 2: Causal Links.

Producer	Condition	Consumer
A_0	ON(B,A)	S_1
A_0	CLEAR(B)	S_1
A_0	ARMEMPTY	S_1
S_1	HOLDING(B)	S_2
A_0	CLEAR(D)	S_2
S_2	ARMEMPTY	S_3
A_0	ONTABLE(C)	S_3
A_0	CLEAR(C)	S_3
S_3	HOLDING(C)	S_4
S_1	CLEAR(A)	S_4
S_2	ON(B,D)	A_∞
S_4	ON(C,A)	A_∞
A_0	ONTABLE(A)	A_∞
A_0	ONTABLE(D)	A_∞

Step 3: Ordering Constraints.

$$A_0 \prec S_1 \prec S_2 \prec S_3 \prec S_4 \prec A_\infty$$

Justifications:

- $S_1 \prec S_2$: S_2 needs HOLDING(B) produced by S_1.
- $S_2 \prec S_3$: S_3 needs ARMEMPTY restored by S_2.
- $S_1 \prec S_4$: S_4 needs CLEAR(A) produced by S_1.
- $S_3 \prec S_4$: S_4 needs HOLDING(C) produced by S_3.

Threat Analysis: S_1 deletes ARMEMPTY, but $A_0 \xrightarrow{\text{ARMEMPTY}} S_1$ uses S_1 as its own consumer — no external threat. S_3 needs ARMEMPTY from S_2 (not A_0); $S_1 \prec S_2$ ensures S_1 cannot intervene between S_2 and S_3. **No unresolved threats.**

Step 4: Why POP Avoids Backtracking.

1. **Least Commitment Principle:** POP does not impose a total order upfront. It adds ordering constraints only when a causal link is threatened. Subgoals ON(C,A) and ON(B,D) are worked on independently.
2. **No Goal Interaction Backtrack:** A total-order planner committing to "achieve ON(C,A) first" would stack C on A. Then achieving ON(B,D) requires unstacking B from A, which destroys the causal link for CLEAR(A). POP detects threats to causal links before execution and resolves them via promotion or demotion — *without restarting the search.*
3. **Explicit Causal Support:** Every precondition has a named producer. If a new action threatens a link, POP either promotes the consumer or demotes the threat. This replaces backtracking with a targeted fix.

2023 — Q5 (9 marks)

Part (a) — STRIPS + Air Cargo Action Schema [4 marks]

2023 Q5(a) — STRIPS + Air Cargo

What is STRIPS? What is the Action Schema of STRIPS? Devise a planning problem for **Air Cargo Transport** (loading cargo, unloading cargo, flying planes between airports).

Solution

STRIPS: See 2024 Q5(a) above.
Air Cargo Transport — Action Schemas:

`Action: LOAD(cargo, plane, airport)`
Preconditions: {AT(cargo, airport), AT(plane, airport)}
Add-effects: {IN(cargo, plane)}
Delete-effects: {AT(cargo, airport)}

`Action: UNLOAD(cargo, plane, airport)`
Preconditions: {IN(cargo, plane), AT(plane, airport)}
Add-effects: {AT(cargo, airport)}
Delete-effects: {IN(cargo, plane)}

`Action: FLY(plane, from, to)`
Preconditions: {AT(plane, from), AIRPORT(from), AIRPORT(to)}
Add-effects: {AT(plane, to)}
Delete-effects: {AT(plane, from)}

Example Problem Instance:
Initial: AT(C1, JFK), AT(P1, JFK), AIRPORT(JFK), AIRPORT(SFO)
Goal: AT(C1, SFO)
Plan: LOAD(C1, P1, JFK) → FLY(P1, JFK, SFO) → UNLOAD(C1, P1, SFO)

Part (b) — POP for Standard Block World [5 marks]

2023 Q5(b) — POP Block World (same problem as 2024)

Block World (same Start / Goal). Solve using partial-order planning.

Solution

The problem is identical to 2024 Q5(b). The minimal POP plan is:

$$A_0 \prec S_1 = \text{UNSTACK}(B, A) \prec S_2 = \text{STACK}(B, D) \prec S_3 = \text{PICKUP}(C) \prec S_4 = \text{STACK}(C, A) \prec A_\infty$$

Causal links and ordering constraints are identical to 2024 Q5(b) — see that section.
POP vs. total-order planning:

- **Total-order** commits to a sequence upfront. If goals interact (one goal's achievement undoes another's precondition), it backtracks and tries a different ordering — potentially exponential search.

- **POP** defers ordering commitments. Goals ON(C,A) and ON(B,D) are treated as independent open conditions. The ordering $S_1 \prec S_4$ (UNSTACK before STACK(C,A)) is only imposed because S_4 needs CLEAR(A) produced by S_1 — not because of an arbitrary total-order guess. No backtracking is needed.

2022 — A-3 (9 marks)

Part (a) — Block World Operator Preconditions [2 marks]

2022 A-3(a) — Preconditions of Block World Operators

Write the preconditions of: UNSTACK(A,B), STACK(A,B), PICKUP(A), PUTDOWN(A).

Solution — Memorise These Verbatim

- **UNSTACK(A, B):** {ON(A,B), CLEAR(A), ARMEMPTY}
  A is on top of B; A has nothing above it; arm is free.
- **STACK(A, B):** {HOLDING(A), CLEAR(B)}
  Arm holds A; B is clear to receive A.
- **PICKUP(A):** {ONTABLE(A), CLEAR(A), ARMEMPTY}
  A is on the table; A has nothing above it; arm is free.
- **PUTDOWN(A):** {HOLDING(A)}
  Arm is holding A.

Part (b) — Block World with STRIPS / POP [5 marks]

2022 A-3(b) — Block World (same problem)

Block World (same Start / Goal). Solve using goal stack planning / partial-order planning / STRIPS.

Solution — Goal Stack / STRIPS Approach

Goal Stack Planning (STRIPS divide-and-conquer):

Step	Action	State after action
0	— (initial)	ON(B,A), ONTABLE(A), ONTABLE(C), ONTABLE(D), ARMEMPTY, CLEAR(B), CLEAR(C), CLEAR(D)
1	UNSTACK(B,A)	ONTABLE(A), ONTABLE(C), ONTABLE(D), HOLDING(B), CLEAR(A), CLEAR(C), CLEAR(D)
2	STACK(B,D)	ONTABLE(A), ONTABLE(C), ONTABLE(D), ON(B,D), ARMEMPTY, CLEAR(A), CLEAR(B), CLEAR(C)
3	PICKUP(C)	ONTABLE(A), ONTABLE(D), ON(B,D), HOLDING(C), CLEAR(A), CLEAR(B)
4	STACK(C,A)	ONTABLE(A), ONTABLE(D), ON(B,D), ON(C,A), ARMEMPTY, CLEAR(B), CLEAR(C)

Goal verification: ON(C,A) ✓ ON(B,D) ✓ ONTABLE(A) ✓ ONTABLE(D) ✓

Why this order? UNSTACK(B,A) must come first because STACK(C,A) requires CLEAR(A), which is blocked by B. After removing B, we immediately place it on D (achieving ON(B,D)) and restore ARMEMPTY. Then PICKUP(C) → STACK(C,A) achieves ON(C,A).

For the POP representation with causal links and ordering constraints, see 2024 Q5(b).

Part (c) — Short Notes: Generative AI [2 marks]

2022 A-3(c) — Generative AI

Write short notes on Generative AI.

Solution

Generative AI refers to AI systems that create new content (text, images, code, audio) statistically resembling their training data.

- **Key Models:** Large Language Models (LLMs like GPT, Claude) predict the next token; Diffusion models (DALL-E, Stable Diffusion) denoise from Gaussian noise; GANs pit a generator against a discriminator.
- **Applications:** Text/code generation, image synthesis, drug discovery, synthetic training data, automated planning.
- **Limitations:** Hallucinations (plausible but false outputs), training data bias, no persistent world-model or memory across sessions, high compute cost.
- **Relevance to AI planning:** LLMs can act as approximate forward planners,

but lack the formal guarantees (soundness, completeness) of STRIPS/POP.

2021 — Q6(b): Sussman Anomaly [4 marks]

2021 Q6(b) — Sussman Anomaly with POP

Block World — **Sussman Anomaly**:

Initial State: C on A, B on table, A on table
⇒ ON(C,A), ONTABLE(A), ONTABLE(B), CLEAR(C), CLEAR(B)

Goal State: A on B, B on C
⇒ ON(A,B), ON(B,C)

Develop an effective and complete plan using POP / STRIPS, addressing threats to causal links and open conditions.

Solution

Why it is the "Sussman Anomaly": No total-order planner can solve this without interleaving subplans.

- To achieve ON(A,B): put A on B. But B needs to be on C first, or A's position on C blocks things.
- To achieve ON(B,C): put B on C. But C currently has A on top of it.
- Goals cyclically interfere — total-order planning backtracks endlessly. POP resolves this via threat detection.

Block World Actions (simplified — no arm required):

A3: Move-C-from-A-to-Table
PRE: {ON(C,A), CLEAR(C)} ADD: {ONTABLE(C), CLEAR(A)} DEL: {ON(C,A)}

A1: Move-B-from-Table-to-C
PRE: {ONTABLE(B), CLEAR(B), CLEAR(C)} ADD: {ON(B,C)} DEL: {ONTABLE(B), CLEAR(C)}

A2: Move-A-from-Table-to-B
PRE: {ONTABLE(A), CLEAR(A), CLEAR(B)} ADD: {ON(A,B)} DEL: {ONTABLE(A), CLEAR(B)}

POP Step-by-Step:
Step 1 — Initial partial plan: $A_0 \prec A_\infty$
A_0 effects: {ON(C,A), ONTABLE(A), ONTABLE(B), CLEAR(B), CLEAR(C)}
A_∞ open conditions: {ON(A,B), ON(B,C)}

Step 2 — Address ON(B,C):
Add action A1. Causal link: $A_1 \xrightarrow{\text{ON(B,C)}} A_\infty$.
A1's open conditions: ONTABLE(B), CLEAR(B), CLEAR(C) — all from A_0:
$A_0 \xrightarrow{\text{ONTABLE(B)}} A_1$, $A_0 \xrightarrow{\text{CLEAR(B)}} A_1$, $A_0 \xrightarrow{\text{CLEAR(C)}} A_1$.

Step 3 — Address ON(A,B):
Add action A2. Causal link: $A_2 \xrightarrow{\text{ON(A,B)}} A_\infty$.
A2's open conditions: ONTABLE(A), CLEAR(A), CLEAR(B).

$A_0 \xrightarrow{\text{ONTABLE(A)}} A_2$, $A_0 \xrightarrow{\text{CLEAR(B)}} A_2$.
CLEAR(A) is **not** in A_0's effects (C is on A) $\Rightarrow$ need action A3.

Step 4 — Address CLEAR(A) for A2:
Add action A3. Causal link: $A_3 \xrightarrow{\text{CLEAR(A)}} A_2$.
A3's open conditions: ON(C,A), CLEAR(C) — from A_0:
$A_0 \xrightarrow{\text{ON(C,A)}} A_3$, $A_0 \xrightarrow{\text{CLEAR(C)}} A_3$.

Step 5 — Detect and Resolve Threats:

- **Threat 1:** A_2 deletes CLEAR(B).
  Causal link $A_0 \xrightarrow{\text{CLEAR(B)}} A_1$ is threatened by A_2.
  A_2 could be placed between A_0 and A_1, deleting CLEAR(B) before A_1 uses it.
  Resolution — Promotion: force $A_1 \prec A_2$ (consumer A_1 before threat A_2). ✓
- **Threat 2:** A_1 deletes CLEAR(C).
  Causal link $A_0 \xrightarrow{\text{CLEAR(C)}} A_3$ is threatened by A_1.
  A_1 could be placed between A_0 and A_3, deleting CLEAR(C) before A_3 uses it.
  Resolution — Promotion: force $A_3 \prec A_1$ (consumer A_3 before threat A_1). ✓

Final POP Plan (ordering):

$$A_0 \prec A_3 \prec A_1 \prec A_2 \prec A_\infty$$

Final action sequence:

$$\boxed{\text{Move-C-from-A-to-Table}} \longrightarrow \boxed{\text{Move-B-from-Table-to-C}} \longrightarrow \boxed{\text{Move-A-from-Table-to-B}}$$

Complete Causal Link Table:

Producer	Condition	Consumer	Note
A_0	ON(C,A)	A_3	
A_0	CLEAR(C)	A_3	*protected:* $A_3 \prec A_1$
A_0	ONTABLE(B)	A_1	
A_0	CLEAR(B)	A_1	*protected:* $A_1 \prec A_2$
A_0	CLEAR(C)	A_1	*(A0 also provides CLEAR(C) to A1)*
A_3	CLEAR(A)	A_2	
A_0	ONTABLE(A)	A_2	
A_0	CLEAR(B)	A_2	
A_1	ON(B,C)	A_∞	
A_2	ON(A,B)	A_∞	

2020 — B-Q6 (estimated 9 marks)

2020 B-Q6 — STRIPS + Block World Planning

[Exact question not captured in question map. Based on 2020–2024 pattern, this question covered STRIPS + Block World.]

Expected Content Based on Pattern

The 2020 paper (Group-B Question 6) most likely asked:

1. **Define STRIPS** (preconditions, add-list, delete-list) + action schema.
2. **Block World problem** — either the Sussman Anomaly style (C on A, B on table → A on B, B on C) or the standard 4-block version.
3. Possibly: forward/backward planning comparison or definition of planning as search.

Key preparation: Be able to write both block world solutions cold:

- **4-block (2022–2024):** UNSTACK(B,A) → STACK(B,D) → PICKUP(C) → STACK(C,A)
- **Sussman Anomaly (2021):** Move-C-off-A → Move-B-onto-C → Move-A-onto-B

Quick Summary — Exam Patterns

What to Memorise for 10 Jun Exam

ALWAYS asked:

1. **STRIPS definition** (2–3 lines): preconditions + add-list + delete-list. Frame problem solved by: only explicit effects change state.
2. **New domain action schema**: Household Robot (2024), Air Cargo (2023), generic (2022). Pattern: define 1–3 actions with PRE/ADD/DEL.
3. **Block World solution** (same start/goal 2022–2024): UNSTACK(B,A) → STACK(B,D) → PICKUP(C) → STACK(C,A).
4. **POP / causal links / ordering constraints** (2024, 2023, 2022): see table above.
5. **Sussman Anomaly POP** (2021): $A_0 \rightarrow A_3 \rightarrow A_1 \rightarrow A_2 \rightarrow A_\infty$.

Two threats in Sussman (memorise exactly):

- A_2 deletes CLEAR(B) $\Rightarrow$ threatens $A_0 \xrightarrow{\text{CLEAR(B)}} A_1 \Rightarrow$ Promotion: $A_1 \prec A_2$.
- A_1 deletes CLEAR(C) $\Rightarrow$ threatens $A_0 \xrightarrow{\text{CLEAR(C)}} A_3 \Rightarrow$ Promotion: $A_3 \prec A_1$.

Operator preconditions (2022 asked verbatim):

- UNSTACK(A,B): ON(A,B), CLEAR(A), ARMEMPTY
- STACK(A,B): HOLDING(A), CLEAR(B)

- PICKUP(A): ONTABLE(A), CLEAR(A), ARMEMPTY
- PUTDOWN(A): HOLDING(A)

POP vs total-order (asked 2024, 2023): POP uses *least commitment* — only orders actions when a causal link is threatened (promotion/demotion). Total-order guesses a sequence and backtracks when goals interfere. POP never needs to restart; it resolves threats locally.